ChatGPT

Building an Autonomous Multi-Agent Trading System

Introduction

Imagine a trading firm where every expert role – from market analyst to portfolio manager – is performed by AI agents working in tandem. The vision is to create a **"farm" of AI agents** that collaborate like a real company's team, autonomously analyzing data, making decisions, and executing trades to maximize profitability. Recent advances in large language models (LLMs) and agent frameworks have made it possible to construct such multi-agent systems, where specialized AI agents communicate, debate, and coordinate actions. In this report, we explore how to build an AI-agent-powered trading operation using several cutting-edge technologies. We will compare frameworks like **Microsoft AutoGen**, **OpenClaw**, and other open-source solutions, detailing how each can be used to implement a cooperative team of trading agents. The goal is an end-to-end autonomous trading system that can operate 24/7 with minimal human intervention (fully "No Human in the Loop"), whether running on-premises or on cloud infrastructure (e.g. GCP), using entirely open-source tools.

We begin with an overview of the problem and desired features, then discuss use cases and draw parallels between a traditional trading firm's operation and our agent-based approach. A high-level architecture is presented to illustrate how multiple AI agents can form a coherent trading workflow. After that, we delve into specific solutions – including AutoGen, OpenClaw, and others – explaining their implementation, step-by-step setup, and how an end-to-end trading scenario would work in each. Finally, we outline future directions and ideas to extend such an AI agent trading system.

Detailed Summary of the Problem

Building an AI-driven trading firm requires solving several complex problems: how to break down the trading process into roles that AI agents can fulfill, how these agents will communicate and coordinate, and how to ensure reliable, profitable decision-making. In a real trading company, various specialists work together – **market analysts** gather information, **strategists** or **portfolio managers** decide on trades, **risk managers** oversee exposure, etc. Recreating this with AI means each of these roles becomes an autonomous agent with access to relevant data and tools. The agents must share information and "agree" on trading decisions in a structured way, much like humans in a meeting.

Key challenges include: ensuring each agent has the *expertise* of its role (which may involve fine-tuned LLMs or specialized prompts), designing a *communication protocol* or conversation mechanism for agents to share insights, preventing chaotic or inconsistent behavior (the agents need a method to reach a consensus or final decision), and integrating with real-world data and trading execution systems. Moreover, since we aim for **no human intervention**, the system must handle continuous operation, error recovery, and dynamic market changes on its own.

In summary, the problem is to create a **coordinated multi-agent AI system** that can perform end-to-end trading activities. This entails solving both *AI orchestration* (how agents think and talk to each other) and *engineering integration* (connecting to data feeds, executing trades via APIs, etc.) in a robust manner. We will see that different frameworks provide different abstractions to tackle these challenges – some focus on simplifying the agent orchestration (communications, conversations), while others provide ready-made autonomous agents with tool integrations.

Features of an AI Agent Trading System

A successful AI agent trading "farm" should exhibit the following features:

- **Specialized Agent Roles:** Each agent has a distinct specialization (e.g. technical analysis, fundamental analysis, news sentiment, geopolitical risk, portfolio optimization, options strategy, etc.). This specialization can be implemented via tailored prompts or fine-tuned models for that domain [1]. For example, a *Technical Analyst Agent* focuses on chart patterns and technical indicators, while a *Geo-Politics Specialist Agent* monitors news and geopolitical events for market impact.

- **Collaborative Decision Making:** The agents don't work in isolation – they communicate through a **multi-agent conversation or shared memory**. They can debate or peer-review each other's findings. For instance, a bullish agent and a bearish agent might argue the merits of a potential trade, providing a balanced view [2]. The system should support such structured dialogue and consensus-building (akin to an investment committee meeting).

- **Autonomous Operation with Tool Use:** The agents should be able to **take actions autonomously** – not just chatting – by using tools or APIs. This includes retrieving real-time data (market quotes, news feeds), running analyses (executing code or calculations), and finally placing trade orders through brokerage APIs. Agents may use an approach like the ReAct framework (Reasoning and Acting) to decide when to invoke tools [3]. For example, an agent might decide to call an API to fetch the latest price of a stock or execute a Python script to run a portfolio optimization routine.

- **Memory and Learning:** The system should maintain memory of past interactions and outcomes. Agents need **long-term memory** to avoid repeating mistakes and to adjust to market regime changes. For example, a *Portfolio Optimizer Agent* should remember recent portfolio allocations and performance. Memory could be stored as conversation logs, embeddings (for recalling similar past scenarios), or even a knowledge base of market facts. Open-source agent frameworks often include memory modules (e.g., vector databases for semantic recall, or simple persistent files for storing state) [4] [5].

- **Risk Management and Constraints:** Just as a real firm imposes risk limits, the AI system must have guardrails. A dedicated *Risk Manager Agent* can monitor the overall exposure and veto or modify trades that violate risk thresholds (e.g., too much leverage or concentration). Additionally, the platform should include **safety mechanisms** to prevent errant behavior – for instance, requiring certain conditions to be met before an agent can execute a high-impact action, or having an oversight logic to catch obviously bad decisions (this is analogous to a human risk officer approving major trades).

- **Real-Time and Continuous Operation:** Markets move quickly, so the agents should operate in (near) real-time and be able to run continuously. This includes scheduling periodic tasks (for example, an agent that every morning gathers pre-market news, or a heartbeat that triggers periodic portfolio checks). An always-on architecture is needed, possibly with a scheduler or event-driven triggers. Some frameworks include built-in schedulers (e.g., OpenClaw has a cron-like system for recurring tasks [6]) to support this.

- **Scalability and Deployment Flexibility:** The system should be deployable on different infrastructures. We target open-source solutions that can run **on-premises or on cloud** (for instance, on a GCP VM or Kubernetes cluster) to allow full control and data privacy. It should scale to additional agents or increased data loads as needed. The communication between agents could be on a single machine process or distributed, but needs to be efficient to handle potentially many messages.

- **Transparency and Monitoring:** While autonomous, the system should provide logs and insight into agent decisions for debugging and trust. (This is an important practical feature – as one blog noted, operating such always-on AI agents without observability can feel like "faith rather than engineering" [7] [8].) Implementing monitoring dashboards or at least detailed logging of each agent's actions and conversations is important for later analysis and compliance (especially in trading, where you need to explain decisions).

These features collectively ensure the agent-farm behaves like a competent trading firm's team: well-informed, collaborative, fast, and safe. Next, we consider specific use cases demonstrating these capabilities.

Use Cases

An autonomous multi-agent trading system can support numerous use cases, including:

- **Daily Market Research and Strategy Formulation:** Each morning before market open, the *Analyst agents* (fundamental, technical, sentiment specialists) gather fresh data – earnings reports, overnight news, technical indicator readings. They produce a report of findings. A *Strategy agent* (or a group debate between a bullish and bearish researcher) then uses these findings to propose trading opportunities (e.g., "Tech stock XYZ is undervalued after earnings – consider a long position"). This mimics a morning research meeting where analysts brief the portfolio manager [9] [2].

- **Automated Day Trading:** During market hours, a *Day Trader Agent* continuously monitors price action and indicators. If a certain pattern emerges (say a breakout or an unusual options activity), it signals a trade. The *Technical Analyst Agent* might confirm the pattern, and a *Decision Maker Agent* decides to execute the trade. This all happens autonomously within seconds. For instance, the day trader agent could use a tool to fetch minute-by-minute data and an LLM to interpret patterns, then call a trade API.

- **Portfolio Rebalancing and Optimization:** A *Portfolio Optimizer Agent* periodically reviews the portfolio's asset allocation against targets and risk metrics. If the portfolio drifts (say too much weight in one sector after a rally), the agent proposes trades to rebalance. It might run an

optimization algorithm (via a Python tool) to find an optimal allocation. The *Risk Management Agent* would review the proposal to ensure it stays within risk limits (e.g., checking Sharpe ratio improvement or drawdown limits). Then trades are executed to adjust the portfolio.

- **News-Driven Trading (Event Trading):** A *News Analyst Agent* watches news feeds and social media for any breaking news (e.g., central bank announcements, geopolitical events). On detecting a significant event, it summarizes the news impact. A *Geo-Politics Specialist Agent* might add context for geopolitical news. Then a *Trader Agent* uses this info to decide on a quick trade (for example, shorting an index on news of a geopolitical conflict). The system thus can react to events faster than a human could.

- **Options Strategy and Hedging:** An *Option Chain Specialist Agent* could monitor options order flow and volatility surfaces. If it detects unusual activity (perhaps heavy put buying indicating hedging by big players), it alerts the *Decision Maker Agent*. The system might then decide to hedge the portfolio or exploit the information (maybe buy protective puts if the flow suggests a downturn). Similarly, the *Option Agent* can construct options strategies (spreads, straddles) as responses to market conditions and pass them to the trader for execution.

- **Multi-Agent Strategy Backtesting:** Beyond live trading, the agents could be used in simulation to develop strategies. One can run the entire multi-agent system on historical data to see how they would have performed (similar to how TradingAgents framework simulated a firm [10]). Agents could "learn" from this by updating their prompts or strategies for better outcomes. This use case straddles into the development phase – using the agents to fine-tune their collaboration before real money is at stake.

These use cases illustrate that an agent-farm can cover the lifecycle of trading: research -> decision -> execution -> feedback. In essence, any task a human trading team does could become an agent's responsibility, from reading earnings reports to executing orders.

How Companies Operate (Relating Roles to Agents)

To design our agent system, it helps to understand a traditional trading firm's structure and then map each role to an AI agent. A typical hedge fund or trading firm might operate as follows:

- **Research/Analysis Department:** Human analysts gather information: fundamental analysts read financial statements and news, technical analysts study price charts, sentiment analysts gauge social media or news sentiment. In our system, each of these can be an agent. For example, a *Fundamental Analyst Agent* reads financial reports or uses an LLM tuned on financial data to output key insights, while a *Sentiment Analyst Agent* might use language sentiment analysis on news headlines. These correspond to the *Analyst Team* in the TradingAgents research, which included agents for fundamentals, news, sentiment, and technical analysis [9].

- **Investment Committee / Strategy Team:** In a firm, portfolio managers or strategists debate investment ideas – often presenting bullish vs bearish cases to avoid bias. We can emulate this with agents adopting contrary viewpoints. In the TradingAgents framework example, they implemented a *Bullish Researcher Agent* and *Bearish Researcher Agent* who debate the merits of an investment [2].

Our system could have a similar pair of agents: one always looking for why a trade **should** be made, another arguing why it **shouldn't**. Their dialogue ensures a balanced consideration. This debate could be facilitated via a multi-agent conversation where each agent's turn is structured (e.g., each presents arguments, possibly moderated by another agent or by the system's logic).

- **Trading/Execution Desk:** Once a decision is made, a trader (or an automated execution system) places orders and manages positions. In our AI firm, the *Trader Agent* fulfills this role. This agent takes the final output of the research and debates and decides specific trade actions (what to buy/sell, how much, timing). It then uses a tool to execute the trade (calling a brokerage API or a simulated exchange). The Trader Agent also can adjust orders if needed (e.g., if price moves or to enforce risk limits passed from the risk agent). In TradingAgents, this corresponds to the *Trader Agents* that execute based on the Analyst and Research team's input [11].

- **Risk Management:** Human risk managers monitor the overall portfolio risk (value at risk, leverage, exposure to factors, etc.) and intervene or advise to keep within limits. We implement a *Risk Management Agent Team* that continuously evaluates the portfolio's risk profile [12]. This could be multiple agents or one agent with a broad mandate. For example, separate risk agents might focus on different risk aspects: one on market risk (volatility, VaR), one on credit/counterparty risk, and one on compliance. In the TradingAgents paper, they mention a dedicated risk management team agent to ensure the portfolio stays within tolerance [12]. In practice, our Risk Agent could listen to the Trade Agent's proposed actions and either approve, modify (e.g., reduce position size), or veto them if they breach limits.

- **Portfolio Manager (Coordinator):** In a firm, a portfolio manager synthesizes all inputs and makes the final call. In our system, this role might be implicit – it could be the *Trader Agent* itself or a separate *Coordinator Agent* that takes the analysts' output and asks the trader to execute. Another design is to have a hierarchical agent: a *Manager Agent* that "manages" the other agents by assigning tasks and queries, somewhat like a human manager directing staff. For instance, a Manager Agent could query the Analyst Agents for updates, query the Risk Agent for clearance, then instruct the Trader Agent to act. This Manager agent would embody the firm's overall strategy and objectives (we might encode goals like "maximize returns with X risk limit" into its prompt so it can weigh inputs accordingly).

Relating these roles to agents clarifies the division of labor in the AI system. We essentially construct a **virtual trading floor**: Analysts (AI) comb through data, specialists provide opinions, and a central decision-maker agent backed by risk oversight makes the call. Each agent's **"job description"** can be encoded in its system prompt or specialized model so that it behaves appropriately (e.g., the Geo-Politics agent's prompt might say "You are an expert in global political events and how they affect markets. Your job is to alert the team about geopolitical news and interpret its impact on currencies and stocks…").

The parallels with human teams help ensure we don't miss critical functions (for example, always include some risk control). By assigning each agent a clear role, we also make the multi-agent system more interpretable and modular.

End-to-End Workflow: How the Project Works

Let's walk through how the entire multi-agent trading system would operate from start to finish in a trading day, integrating everything:

1. **Morning Initialization:** Suppose it's 6am and markets will open in a few hours. The system's scheduler or *heartbeat* triggers a sequence to prepare for the day. The *Manager/Coordinator Agent* (or simply a predefined automation script) prompts the *Analyst Team agents* to gather data. For example, the Coordinator might send a message: "Gather pre-market insights for today." The Fundamental Analyst Agent pulls overnight news on companies (using a news API tool), the Technical Analyst Agent calculates technical indicator states, the Sentiment Agent gathers social media sentiment. Each agent returns a summary of their findings.

2. **Synthesis and Debate:** The collected analyses are forwarded to the *Research/Strategy Agents*. Perhaps the system triggers the *Bull vs Bear agents* conversation: The Bullish Research Agent receives all the analysts' data and argues in natural language "Today looks positive because X, Y, Z". The Bearish Agent responds with caution: "However, we should be wary of A, B, C." They might go back and forth for a couple of turns, potentially asking an Analyst Agent for clarification if needed. This debate is essentially the AI equivalent of an investment committee meeting. The conversation is managed either by the framework's conversation orchestrator or by a controlling logic that ensures each agent speaks in turn and references the data (some frameworks like AutoGen allow setting conversation rules so agents speak in a specified order [13] [14]).

3. **Decision Formulation:** After the debate, the *Trader/Decision Agent* is prompted to make a decision. It receives the arguments and possibly direct recommendations from the research agents. Its prompt might be: "Based on the above analysis, propose a set of trades (with rationale)." The Trader Agent might output something like: "**Trade Proposal**: Go long 100 shares of XYZ, as fundamentals are strong and technicals show an uptrend. Short 50 shares of ABC to hedge sector risk. No action on DEF due to conflicting signals." This answer is produced by the Trader Agent's LLM after digesting the others' messages.

4. **Risk Check:** Before execution, the *Risk Management Agent* automatically evaluates the Trader's proposal. It might check each proposed trade against risk limits (perhaps using rules or computing metrics via a tool). For instance, if XYZ and ABC are highly correlated, the risk agent might flag that the hedge is inadequate or that the position size is too large for the portfolio value. The Risk Agent then gives its assessment: e.g., "XYZ position is within risk limits, ABC short is acceptable, overall VaR is within range. Proceed." If something is off, the Risk Agent could modify the proposal: "Reduce XYZ to 80 shares to stay within exposure limit." This step ensures an independent check – similar to how a risk officer must approve trades above a threshold.

5. **Execution:** Once approved, the system moves to execution. The *Trader Agent* (or an Execution sub-agent/tool) connects to the trading API. It sends the orders (buy/sell) and then confirms execution. In many frameworks, this action is performed via a **tool use** – for example, a code tool that runs a Python script using a trading SDK, or an HTTP request tool to a brokerage API. If using OpenClaw, a "skill" for trading can be installed which knows how to place orders [15]. The Trader Agent might output via the chat something like "Executing order: BUY 100 XYZ at market" as it does so, for logging.

6. **Monitoring and Feedback:** Throughout the day, agents continue to monitor. The Price Action Specialist Agent might watch intraday movements: if a trade is going badly (say XYZ falls 5%), it could alert the Trader Agent to reconsider (stop loss or adjust). The system's continuous **heartbeat** can trigger periodic checks (like "Every 30 minutes, update on portfolio performance") [16]. Agents update their memory – e.g., the Memory store records that a trade in XYZ was made and at what price. If any unexpected event occurs (breaking news at noon), the News Agent will kick off another mini-cycle of analysis and possibly new trades.

7. **End-of-Day and Reporting:** After market close, the system can do an end-of-day routine. The agents log the day's trades and outcomes. The Portfolio Optimizer might run once more to suggest any overnight risk hedges. A *Report Agent* could even compile a summary of the day's performance to send to the human (if any) or just log it. This could include P/L, what decisions were made and why – providing transparency. Because all agent communications are essentially text, it's straightforward to compile these into a human-readable report for audit or review.

8. **Continuous Learning (optional):** Over time, the system could improve. For instance, if certain agents frequently make errors or if the Risk Agent finds itself correcting the same type of issue, developers might update the prompts or logic. One could incorporate a reinforcement learning loop (outside the scope of current frameworks out-of-the-box, but a possibility in future) where the system learns from trading outcomes to adjust strategies.

This end-to-end flow shows the **entire project in motion**: multiple agents, each doing their part, handing off information and tasks, and collectively managing a trading strategy. It's a complex orchestration, but modern agent frameworks are explicitly designed to facilitate such structured multi-agent workflows [17] [1].

Detailed Architecture Diagram

[18]

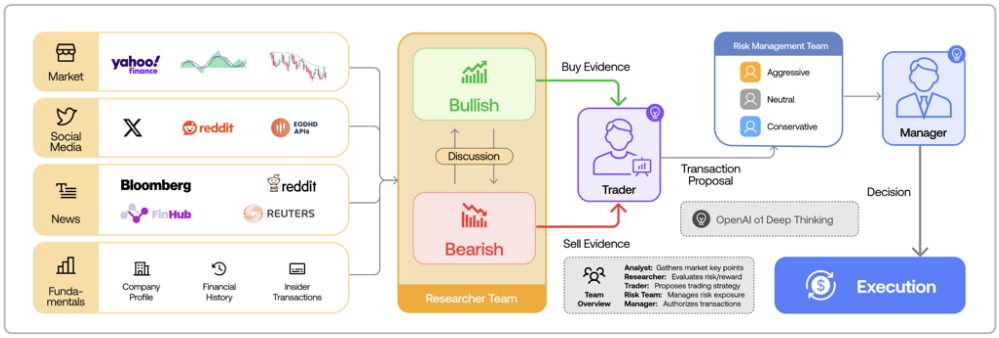


Illustration: High-level architecture of a multi-agent trading system (based on the TradingAgents framework). Specialized analyst agents feed information to researcher agents (bullish vs bearish) who debate and produce insights. A trader agent formulates trade actions, which are vetted by risk management agents. The manager/coordinator (or the trader agent itself) then executes trades via a market interface. Arrows indicate the flow of information and decisions.

The above diagram (inspired by TradingAgents [19]) visualizes the architecture:

- On the **left**, various data sources (market data, news feeds, social media, financial reports) are accessed by the Analyst agents. These agents process raw data into summaries or signals.
- The **Research team** (Bullish vs Bearish agents) takes those inputs and engages in a structured dialogue (the "Discussion" bubble in the diagram). They produce arguments for and against potential investments, mimicking a committee discussion.
- The outcome (buy/sell rationale) flows to the **Trader agent**, which creates a concrete *Transaction Proposal* (specific orders to execute).
- The proposal goes through the **Risk Management team**. In the diagram, multiple risk profiles (Aggressive, Neutral, Conservative) are considered – this could be implemented as multiple risk agents or a single risk agent that can simulate different risk appetites. They ensure the proposal aligns with risk constraints.
- Finally, the decision is ratified by a central **Manager** agent (or directly by the Trader if we simplify) and then sent to **Execution**, which interfaces with the real trading system (brokers/exchanges).

The architecture also shows feedback loops: for example, the risk team might send feedback back to the trader (to adjust the proposal), or the trader might query analysts for additional data if needed. These interactions are facilitated by the multi-agent framework's ability to route messages between agents properly.

This design emphasizes modularity – one can add more analyst agents for new data types (e.g., an agent that analyzes blockchain data if trading crypto), or add more trader agents if trading different asset classes (equities, forex, etc.) concurrently, each still overseen by risk management.

Next, we will explore specific **implementations** of such an architecture using different technologies. We compare how frameworks like AutoGen, OpenClaw, LangChain, and the OpenAI Agents SDK each approach the building of this multi-agent system, including technical diagrams and steps for each.

Technology Solutions Overview

There are several technologies and frameworks that can be used to build multi-agent AI systems. Each has its strengths and ideal use cases. Below is a brief overview of the notable ones in 2025-2026, before we dive into detailed implementations:

- **Microsoft AutoGen:** An open-source Python framework by Microsoft for orchestrating multi-agent conversations [17]. AutoGen provides abstractions for creating agents (LLM-backed or tool-using) and letting them converse, debate, or collaborate to solve tasks. It's like a toolkit to build the kind of structured dialogues our trading agents need, with support for code execution and custom conversation rules. AutoGen has been used in complex workflows and was even merged into Microsoft's larger "Agents Framework" as it matured [20].

- **OpenClaw:** A viral open-source autonomous agent platform that runs continuously on your hardware [21]. OpenClaw is more of a **ready-to-run AI "employee"** than a coding framework – you install it, configure some skills and prompts, and you get an agent (powered by an LLM like Anthropic's Claude) that can do tasks on its own (from browsing web to running shell commands) [22]. It supports connecting to messaging apps (so you can chat with it or have agents chat together

in a group) and scheduling tasks [16]. It's not a Python library but a standalone system primarily configured via Markdown files (AGENTS.md, TOOLS.md, etc.), emphasizing ease of use (no coding) in exchange for some flexibility [23] [24].

- **LangChain and LangGraph:** LangChain is a popular Python library for building LLM applications, and LangGraph (introduced in 2024) is an extension focusing on multi-agent orchestration within LangChain [25]. LangGraph allows you to create directed graphs of agents, supporting **stateful interactions, sequential or parallel workflows, and memory** [26]. It integrates with the broader LangChain ecosystem (vector stores, tools, etc.) and has seen adoption in enterprise settings, e.g., powering customer support bots and data analysis assistants [27]. Essentially, LangChain/LangGraph provide building blocks to "code" your multi-agent system with fine control.

- **OpenAI's Agents SDK:** Released by OpenAI in 2025, this is a lightweight framework for multi-agent workflows in Python [28]. It's designed to be provider-agnostic (works with many LLMs) and emphasizes tracing, guardrails, and ease of use. You can think of it as OpenAI's answer to AutoGen – enabling developers to spin up multiple agents that can talk to each other or handle sub-tasks, without tying to a specific vendor. It gained popularity due to OpenAI's backing and simplicity, useful for quickly prototyping agent workflows [29].

- **CrewAI:** Another open-source framework (Python) that appeared in 2024, focusing on "role-playing" agents with minimal setup [30]. It's independent of LangChain, providing a simpler interface to define agents and their roles, then orchestrate conversations. CrewAI became popular in scenarios like customer service or marketing content generation, where you might have a few agents each with a specific role (e.g., a customer agent, a support agent) chatting. It requires some coding but tries to reduce complexity of multi-agent orchestration [31].

- **Others:** There are additional solutions like **Google's Agent Development Kit (ADK)** which is tied into Google Cloud and their models (Gemini, Vertex) [32], **Dify** which is a low-code platform for creating agents visually [33], and **Mastra** which is a TypeScript-based agent framework for JavaScript developers [34]. There have also been experimental projects like AutoGPT and BabyAGI (early autonomous agent demos in 2023), but these have largely been supplanted by more robust frameworks – indeed users found AutoGPT/BabyAGI great for demos but not production-ready [35]. For our purposes (on-premise, code-oriented, open source), we will focus on Python-based, developer-friendly frameworks: AutoGen, OpenClaw, LangChain/LangGraph, and OpenAI's Agents SDK.

Each of these approaches the problem differently: **AutoGen/LangChain/OpenAI SDK** require you to write code to define and orchestrate agents (giving maximum flexibility and integration capability), whereas **OpenClaw** provides a lot out-of-the-box (agent runtime, messaging integration, scheduling) but is less of a coding library and more a configured platform.

In the next sections, we will examine each selected solution in detail, including how to implement our trading firm scenario step-by-step and an end-to-end example of it in action. We'll use Python for the code examples (except OpenClaw, which uses configuration and Node.js runtime but we'll keep instructions high-level). All solutions are open-source and can be deployed locally or on cloud servers. We will avoid any proprietary or purely UI-driven tools – the focus is on coding frameworks suitable for developers.

Solution 1: Building with Microsoft AutoGen

Implementation Overview

Microsoft's **AutoGen** framework is well-suited for constructing a team of LLM-driven agents that converse to solve tasks [17]. It essentially provides a programmable way to create agents (each backed by an LLM like GPT-4 or others) and define how they interact – either in a freeform conversation or following certain patterns. Key capabilities of AutoGen include support for **multi-agent conversations, tool usage (like code execution)**, the ability to have agents that are AI or humans in the loop, and flexible conversation patterns (one-to-one, one-to-many, many-to-many, hierarchical) [36] [37].

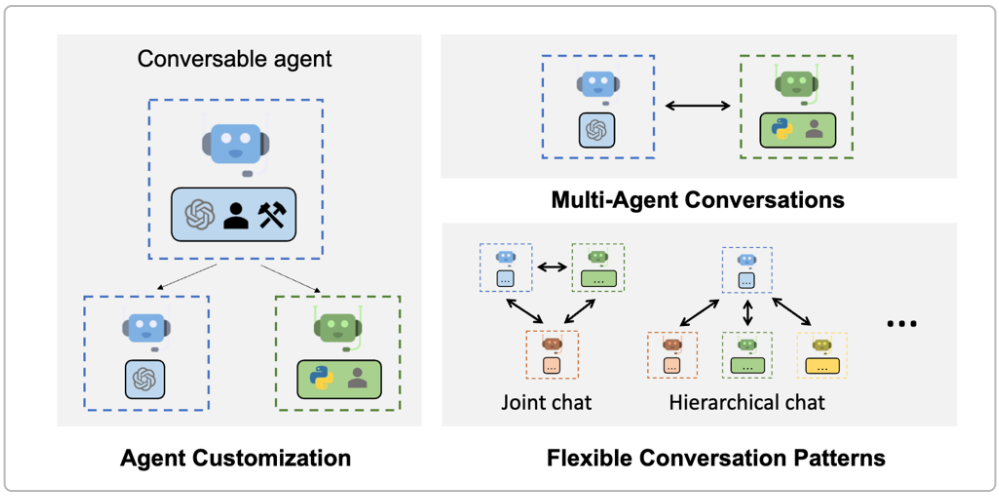
In the context of our trading system, AutoGen acts as the **orchestration backbone**: we will use it to create specialized agent instances for each role (Analyst, Trader, Risk, etc.) and script their interaction sequence. AutoGen allows for event-driven chats – meaning an agent can produce output which can be programmatically inspected and then passed to another agent. This is crucial to implement structured workflows. For example, we can use AutoGen's **GroupChat** feature to have multiple agents in a shared conversation, or we can drive the interaction via a custom loop that sends a message to Agent A, then to Agent B, etc., depending on the state.

One advantage of AutoGen is that it comes with **built-in patterns and examples**. The developers have demonstrated scenarios like one agent writing code and another reviewing it, or an agent chain for research where one agent fetches data and another analyzes it [38] [39]. We can leverage similar ideas: for instance, one agent could write a piece of Python code to fetch stock data, and another agent could execute it and return results (AutoGen directly supports agents that can write and run code in a sandbox) [40] [41]. This means our Technical Analyst agent might not need a pre-built tool – it can dynamically generate code to compute an indicator, run it, and get the output, all under AutoGen's management [42].

AutoGen's architecture can be visualized as a mediator managing the conversation between agents and tools. Each **AssistantAgent** (AI agent) has an LLM configuration (you can use OpenAI models, local models, etc.) and optionally a profile (like a system prompt defining its role). There is also a concept of a **UserProxyAgent** which is a stand-in for a human user – but in our case, we might use a UserProxyAgent as a driver agent that initiates the conversation among the AIs (essentially we automate the user input as well). In multi-agent chats, AutoGen can let agents talk to each other for a set number of turns or until a termination condition is met [43] [37].

Technical Diagram (AutoGen): The figure below illustrates how we set up the agents and conversation in AutoGen for the trading system.

[17]



Conceptual architecture with AutoGen: We instantiate multiple AutoGen agents (LLM-backed), each with a role prompt (Analyst, Trader, Risk). A GroupChat or orchestrator controls the sequence of conversation. Agents can invoke tools (like code execution or web access) through AutoGen's built-in executors. The flow is event-driven: e.g., the Analyst agent's output triggers the Trader agent's turn, and so on, ending with trade execution.

In summary, AutoGen provides a robust foundation to implement complex, multi-step dialogues and tool interactions. It has proven itself in production scenarios where reliability is key [44] [45]. By using AutoGen, we benefit from a lot of built-in functionality (conversation handling, multiple model support, caching, etc.) without having to code the low-level agent loop from scratch.

Step-by-Step Implementation with AutoGen

Implementing the AI agent trading firm with AutoGen involves the following steps:

1. **Environment Setup:** Ensure you have Python installed (3.8+). Install the AutoGen package via pip. For example: `pip install autogen-agentchat` (version 0.2 or latest) [46]. Also gather API credentials for any LLMs you plan to use (OpenAI API key, etc.). This will allow the AutoGen agents to actually generate text via the chosen model. Optionally, set up any required tools (for instance, if using code execution, having a Python environment or Docker for sandboxed execution is recommended).

2. **Define Agent Roles:** Using AutoGen's classes, create an agent instance for each role in your system. AutoGen's primary agent class is `AssistantAgent`. We instantiate, for example:

3. `analyst_agent = AssistantAgent("analyst", llm_config={...})`
4. `trader_agent = AssistantAgent("trader", llm_config={...})`

5. `risk_agent = AssistantAgent("risk_manager", llm_config={...})` Each agent is given a name (identifier) and an LLM configuration. In the `llm_config`, we specify which model to use (e.g., GPT-4) and the API key or other parameters [47] [48]. We will also supply a **system prompt** for each agent that describes its persona and task focus. AutoGen allows passing a `system_message` when creating an agent to set its initial context. For example, the analyst's system prompt might be: "You are a market analyst AI specialized in analyzing financial news and technical indicators." The

trader's: "You are a trading agent responsible for making final trade decisions based on input from others and considering risk." The risk agent's: "You monitor portfolio risk and ensure trades stay within limits."

6. **Enable Tool Usage (if needed):** If we want our agents to use tools (like running code for data or calculations), we configure that. AutoGen supports code execution via `code_execution_config` when creating a UserProxyAgent or AssistantAgent. For instance, we might allow the analyst agent to execute Python code for fetching data. We could do:

```
from autogen import AssistantAgent, UserProxyAgent
from autogen.coding import LocalCommandLineCodeExecutor
analyst_agent = AssistantAgent("analyst",
                llm_config=llm_config_for_gpt4,
                system_message=analyst_prompt)
analyst_agent_instrumented = UserProxyAgent("analyst_proxy",
                code_execution_config={"executor":
LocalCommandLineCodeExecutor(work_dir="workspace")})
```

However, another pattern is to use a separate *proxy agent* to initiate and capture code execution. AutoGen examples show a `UserProxyAgent` can initiate chat and if configured with `code_execution_config`, it will execute any code the AI tries to run [49] [50]. So an alternative is to have the conversation between a `UserProxyAgent` and one or more `AssistantAgent`s, and the proxy handles code tool use. For simplicity in our trading scenario, we might not need a proxy for each – we can give the Trader agent access to, say, a `LocalCommandLineCodeExecutor` to call a trading API script.

7. **Orchestrate Agent Interaction:** AutoGen offers two main ways to coordinate multiple agents:

8. **GroupChat API:** You can create a `GroupChat` object with a list of agents and then call methods to have them converse. By default, group chat might just round-robin or auto decide who speaks, but AutoGen allows customizing the **speaker selection function** [51] [14]. We will likely implement a custom speaker policy to enforce our desired order: e.g., Analyst speaks first, then Trader, then Risk. The GroupChat can be configured with that logic. Once set up, you call something like `groupchat.run(max_steps=some_number)` and AutoGen will handle passing messages between the agents until termination. Termination could be when the Trader agent signals that a decision is made and executed.
9. **Manual Orchestration:** Alternatively, we can manually drive the conversation. For example, use the `UserProxyAgent` to send a message to the analyst agent like "Please provide analysis." Capture the analyst's response, then send that (or a summary of it) to the trader agent, etc. This can be done by calling each agent's `.step()` or using `initiate_chat` in stages. AutoGen provides methods to get the content of an agent's last message which we can forward. This approach is more verbose but gives explicit control at each step (useful if we want to inject custom logic between turns).

For a robust solution, the **GroupChat with custom speaker selection** is ideal since it formalizes the workflow. We will implement a speaker-selection function (similar to a state machine) with states like: INIT -> ANALYSIS -> DECISION -> RISK -> END [13]. For instance:

```
def speaker_selection(last_speaker, groupchat):
    if last_speaker is None:
        return analyst_agent  # start with analyst
    if last_speaker.name == "analyst":
        return trader_agent
    if last_speaker.name == "trader":
        return risk_agent
    if last_speaker.name == "risk_manager":
        return None  # end conversation after risk manager speaks
    return None
```

This pseudo-code indicates: Analyst speaks, then Trader, then Risk, then stop. We would pass this to GroupChat:

```
chat = autogen.GroupChat([analyst_agent, trader_agent, risk_agent],
                         speaker_selection_method=speaker_selection)
chat.run(max_rounds=4, initial_message="Begin trading discussion.")
```

With this, the conversation will proceed in the order we defined, each agent responding in turn. We must ensure that each agent's prompt and the conversation context lead them to do the right thing (the analyst will output analysis, trader will output trade decision, risk agent will output approval/modifications).

1. **Integrate Data Sources and Execution:** The agents need actual data to work with and a way to execute trades. There are a couple of ways:
2. **Incorporate data via prompt or tool:** For example, the Analyst agent could have access to a tool function that fetches data when asked. In AutoGen, one can define a custom function and allow the LLM to call it (similar to OpenAI function calling but handled via agent design). Alternatively, the analyst agent can be prompted with up-to-date info already. We might pre-fetch key data (prices, news) using Python and include it in the prompt ("Context: stock XYZ last close price, key headlines: ..."). For a more dynamic approach, use the code execution: the analyst agent could literally respond with Python code to retrieve data given our prompt encourages it, then the AutoGen executor will run it.
3. **Trade execution**: The Trader agent can use code execution to call a trading API. For instance, if using a paper trading API, we could have a small Python snippet in the conversation like `# Execute trade\nimport requests\nrequests.post("https://broker.api/order", data={...})`. If AutoGen's execution is enabled, it will run this. We should obviously be careful and perhaps use a simulation or sandbox for safety while developing.

4. AutoGen also supports a simpler "tools" concept where an agent message can request a tool and the system executes it, but customizing that might require more coding. Using the code executor as a generic tool is often easiest – the LLM just has to output a code block to do what it wants.

5. **Run the System:** After setup, we initiate the process. For example, we use a `UserProxyAgent` or a simple initial message in GroupChat to kick off the day's analysis as described. Then let the conversation run. We would wrap this in a script that perhaps loops or waits for triggers (like schedule triggers). Since AutoGen is just Python code, we can integrate it with scheduling (e.g., a

cron job or just an infinite loop with sleep intervals in the script to simulate the heartbeat that triggers every X minutes). During runtime, AutoGen will manage the back-and-forth messaging. We should log or print out each agent's outputs for transparency. We might also implement conditions: e.g., if the trader agent's proposal is final and risk agent approves, break out of the conversation loop and not wait for max rounds.

6. **Deployment:** Running on-prem or on GCP is straightforward – it's a Python program. For on-prem, ensure the machine has access to the LLMs (likely via API calls to OpenAI or using Azure OpenAI if on corporate cloud). For GCP, you could deploy this as a container on a VM or Cloud Run. Because AutoGen is just code, it doesn't impose infrastructure requirements; you just need to handle secrets and possibly use VPN if accessing a broker API securely. Optionally, you might use GCP's Vertex AI if you have Vertex endpoints for models; AutoGen can be configured to use non-OpenAI models by customizing the `llm_config` (e.g., pointing to a local model or other provider [52]).

End-to-End Example with AutoGen

To illustrate how AutoGen ties everything together, let's simulate (at a high level) an example conversation flow between agents in a trading scenario:

- **Initial Trigger:** We call the `GroupChat.run()` at morning open with an initial message: "Market open. Analyst, please provide the latest market analysis." This is given by the system to the group. The speaker selection then chooses the Analyst agent to respond first.

- **Analyst Agent Response:** (Backed by an LLM like GPT-4, possibly with some tool use)

- It might output: *"Analyst: Good morning team. Overnight news: XYZ Corp beat earnings expectations, stock is +5% pre-market. ABC Corp got downgrades due to regulatory issues. Technicals: S&P500 momentum is positive, RSI ~65 (approaching overbought), and VIX is low at 15. Sentiment on social media for tech sector is bullish. Key risks: Fed meeting tomorrow could cause volatility."*
- Additionally, if we enabled code execution, the analyst might have actually fetched some data. For example, part of its answer could include a code block it executed: perhaps it ran a quick script to pull current futures data to state "S&P500 +1% pre-market".

- AutoGen returns this message and now passes control (via our speaker function) to the Trader agent.

- **Trader Agent Response:**

- With the analyst's info in context, the Trader agent (LLM) formulates a plan: *"Trader: Based on the analysis, I propose we take a long position in XYZ (since earnings are strong and momentum is up). Also, consider shorting ABC given the downgrades, to hedge sector risk. Suggest buying 100 shares of XYZ and shorting 100 shares of ABC. Stop-loss at 5% loss, take-profit at 10% gain on XYZ. We'll keep ABC short until its issue resolves or it drops 8%."*

- The Trader agent basically created a mini strategy. This gets recorded as the trader's message.

- **Risk Agent Response:**

- Now AutoGen triggers the Risk Management agent. This agent sees the proposals. Its LLM might evaluate the risk: *"Risk Manager: The proposed trades would result in a net long exposure roughly equal to 2% of portfolio – that's within limits. The hedge (ABC short) balances some risk in tech sector. Check: XYZ volatility is moderate; a 5% stop is about $500 risk on 100 shares (acceptable). The positions are within our risk budget. I approve this strategy. Ensure to monitor Fed meeting news that could affect these positions."*

- If something was not okay, the Risk agent could instead respond with modifications (the LLM could be prompted to act conservatively, e.g., "if any trade is too risky, suggest a smaller size").

- **Trader Execution (Tool Use):**

- After the Risk agent, according to our sequence, we had set to end the chat (speaker_selection returned None). However, we might actually want the Trader to perform the execution and perhaps confirm. We could alter the sequence to give Trader one more chance to act after risk approval. For simplicity, assume the Trader now executes:
- AutoGen allows the Trader agent to output an action. If we configured a proxy agent with code execution, the Trader could output something like:

  ```
  [Trader executes trade]
  ```

  (This might be a special syntax or simply the content triggers our script to execute.) Or the Trader could output a JSON or command that our surrounding code interprets as "execute this trade via API". Since AutoGen doesn't inherently know how to place a trade, this part relies on our integration. We might have given Trader agent a function (via the `tool` interface or just by convention) like `TRADER.execute_order(ticker, qty, action)` that we parse and run externally.

- Assuming the trades are executed, we log the execution.

- **Conversation End:** The system might then compile a summary message like: "Executed: Bought 100 XYZ @ $X, Shorted 100 ABC @ $Y. Monitoring ongoing." and then terminate until the next trigger or event.

This example shows how, in natural language form, the agents moved from analysis to decision to risk check to execution – all within an orchestrated AutoGen-managed session. In practice, you'd be monitoring these agent messages in the console or logs. Thanks to AutoGen, we didn't have to implement the low-level message passing – we defined agents and a sequence, and it handled the rest (model calls, enforcing turns, gathering all context from previous messages so each agent sees the full conversation history).

Notably, AutoGen would handle conversation memory for us (each agent's output is part of the group chat history that the next agent sees). This ensures, for example, the Trader agent's LLM can see exactly what the Analyst said. AutoGen also supports long context and can integrate retrieval augmentation if needed [52], which could be useful if we want to inject relevant past data (like "last week's performance" from a database) into the conversation.

Overall, using Microsoft AutoGen, we achieve a coded yet high-level approach to multi-agent orchestration, which is perfect for our needs. We now have a blueprint for an autonomous trading team running via

AutoGen. Next, we'll examine a very different approach: **OpenClaw**, which tries to deliver autonomous agents with minimal coding.

Solution 2: Building with OpenClaw

Implementation Overview

OpenClaw is an open-source autonomous AI agent platform that gained massive popularity for its ability to actually *do things*, not just chat [21]. Unlike AutoGen, which is a developer framework, OpenClaw is a standalone "AI agent runtime" you install and run as a service on your machine [53]. It's designed to act like a personal AI assistant (or a team of assistants) that can connect to your messaging apps (WhatsApp, Telegram, Slack, etc.) and execute real tasks on your computer or online. In essence, OpenClaw turns an LLM (typically Anthropic's Claude, based on its origin as "Clawdbot") into a **persistent, tool-using agent** with memory, multi-channel communication, and scheduling.

For our trading system, OpenClaw can be harnessed to create an always-on agent (or multiple agents) that collectively perform the trading tasks. A key difference is that OpenClaw doesn't require writing Python code to orchestrate conversations – instead, you configure agent behavior through text files: - **AGENTS.md** – defines one or more agents and their instructions. - **TOOLS.md** – defines what tools the agent can use (shell commands, APIs, etc.). - **SKILLS** – individual skills as markdown or scripts that the agent can invoke for specific tasks (placed in a skills directory). - **MEMORY.md** – stores long-term info the agent should remember. - **HEARTBEAT.md** – instructions for periodic tasks the agent should do on its own schedule [4] [16].

OpenClaw essentially **bundles the core architecture needed for an autonomous agent**: - It has a **Gateway** server (Node.js based) that manages the messaging interfaces and agent sessions [54]. - It implements the "agent loop": when a message or trigger comes in, the agent considers it (along with context), produces an LLM query, which might include tool requests (OpenClaw intercepts special syntax to run tools), executes them, then returns a response [55]. This loop can iterate multiple times to complete a task that needs several tool calls [56]. - **Memory** is persisted to disk (no separate DB by default; markdown files for knowledge, plus a SQLite for vector search of past conversations) [4] [57]. - It supports **multi-agent sessions** such that you can run more than one agent persona simultaneously (for example, different chat threads can correspond to different agents) [58]. Agents can even invoke each other via messaging if configured to do so (though this is more indirect than in AutoGen).

What makes OpenClaw appealing is that it comes with a lot of **pre-built skills** (over 700 community-contributed skills in "ClawHub") covering things from email handling to **crypto trading** [59]. For our trading project, this means we might not have to write all tools from scratch – if someone already made a skill to fetch stock prices or execute a trade on Binance, we could install and reuse that. OpenClaw can even automatically fetch new skills when needed by parsing the agent's request and searching its registry [59].

However, orchestrating multiple specialized agents in OpenClaw is a bit different. OpenClaw is primarily one agent per session. But one could create a "council" of agents in a group chat. Indeed, some users have run multiple OpenClaw agents together by making a group chat where each agent is a member and they can mention each other [60] [61]. One agent can delegate tasks to others (one being the main coordinator). This was demonstrated by a user who configured a 3-agent system (a main assistant delegating to two specialized agents) purely through OpenClaw config [62] [60]. In that scenario, all agents operate via the

same Gateway, and because OpenClaw supports multi-channel, if you put them in one channel (like a Telegram group), they effectively converse.

So, our approach with OpenClaw would be: - Use one OpenClaw agent as the **Chief Trading Agent** (like the manager/trader). - Configure additional specialized skills or sub-agents within either the same agent's configuration or separate sessions. We might not easily spin up multiple fully separate agents that talk, but we can simulate specialization via skills. For example, we can create a skill called "TechnicalAnalysis" that when invoked, triggers the agent to analyze technical indicators (we can implement it as a script using TA-Lib or similar). Another skill "NewsAnalysis" could fetch and summarize news. The agent can be instructed in its prompt (AGENTS.md) to use these skills when appropriate (OpenClaw's instructions can include when to activate certain skills based on user queries or context). - Alternatively, run multiple OpenClaw instances or sessions: one for analyst, one for trader, etc., but that gets complex in synchronization. A simpler method: treat OpenClaw as a single composite agent that does the job of coordinating sub-tasks.

OpenClaw's strength is executing tasks autonomously on schedule. For trading, we can schedule regular market checks. Using **HEARTBEAT.md**, we instruct the agent something like: "Every 30 minutes, check for any new market-moving news or price changes and evaluate our portfolio. If significant changes, decide on adjustments." The agent will wake up every interval and do this, without any user prompt [16].

Security side note: OpenClaw is powerful (it can run shell commands, control browser, etc.), which is both a feature and a risk. In a trading context, one must secure API keys and ensure the agent doesn't run arbitrary harmful commands (especially if connected to internet). Experts recommended running OpenClaw in isolated environments due to potential vulnerabilities [63] [64]. But since our scenario is presumably a controlled setup, we just keep this in mind.

Step-by-Step Implementation with OpenClaw

Implementing the system in OpenClaw will involve configuration rather than a lot of coding:

1. **Install OpenClaw:** OpenClaw requires Node.js (v22+). We would install via npm: `npm install -g openclaw@latest`. Then run an onboarding: `openclaw onboard --install-daemon` [65]. This sets up the OpenClaw Gateway as a background service on your machine. During onboarding, it will ask for an LLM provider (likely you'd input your Anthropic Claude API key or OpenAI key; Claude is recommended for long conversations as per docs [66]). It will also prompt to connect messaging channels – e.g., you can link a Telegram bot or Slack bot so that you have a chat interface. For our purposes, we might set up just a **local web dashboard** which OpenClaw provides at http://localhost:18789 for controlling the agent [67]. We might not need user chat at all if we use heartbeat automation, but it's useful for debugging or issuing commands.

2. **Configure the Agent (AGENTS.md and SOUL.md):** After install, the agent's workspace is typically `~/.openclaw/workspace`. We edit the `AGENTS.md` file to define our agent's core instructions. This file can specify the agent's name, personality, and capabilities. For instance:

```
# TradingAgent
Role: Trading AI Agent
Description: You are an autonomous trading agent managing a portfolio. You
```

```
can analyze market data, form trading strategies, and execute trades. You
have specialized skills for technical analysis, news analysis, and options
analysis. Always observe risk management guidelines.
Goals:
- Maximize portfolio returns while controlling risk.
- Communicate your analysis and actions clearly.
- Use skills and tools to get data and execute trades.
```

The exact syntax in AGENTS.md might involve sections for goals or examples of usage. The `SOUL.md` can be used to define more of the agent's "personality" or constants about its behavior (like its risk tolerance or style – e.g., "risk profile: moderate").

We will include an instruction that the agent should effectively play the roles of our team internally. We might not explicitly have multiple separate agents, but we can instruct it like: "When making decisions, consider fundamental, technical, sentiment factors (you have skills to gather each). Debate pros and cons if needed before finalizing a trade." This prompt engineering nudges the single agent to internally mimic multiple viewpoints, which with a powerful model like Claude can work since it can carry multi-threaded reasoning.

1. **Define Tools and Skills (TOOLS.md and skill files):** In TOOLS.md, we declare what tools the agent can use. OpenClaw has built-ins for Shell, Browser, Webhooks, etc. [22]. We'll likely use:
2. Shell: to run custom scripts (Python or bash) – useful for data retrieval or running analytic code.
3. Web (Browser control): possibly to scrape a webpage (like a news site or stock info if needed).
4. API calls: We might register an API integration. For example, if we have a brokerage API, we could set it up as a webhook or write a shell script that calls it.

Example `TOOLS.md` entry:

```
Tool: stocks
Command: python3 ./skills/get_stock_price.py "{ticker}"
Description: Retrieves the current price for a given stock ticker.
```

This would allow the agent to call `stocks("AAPL")` in conversation and OpenClaw will execute get_stock_price.py with AAPL, returning the output to the agent's context.

Next, we create **skills**: - `skills/get_stock_price.py`: a Python script that calls some API (Yahoo finance or a brokerage data API) to get live price. - `skills/news_summary.py`: perhaps uses an API or web scrape to get latest news and then summarize (could even call an LLM for summarization or use a smaller local model). - `skills/execute_trade.py`: a script that sends an order to our trading platform (e.g., using Alpaca API or Interactive Brokers API). We'll need to carefully secure keys (store them in env vars which OpenClaw can pass to scripts). - We can also create Markdown-based skills for instructing behaviors. For example, a skill file `skills/TECHNICAL_ANALYSIS.md` that contains instructions: "If asked about technical analysis, check moving averages, RSI, MACD using price data." Actually, a more standard approach: have the technical analysis skill be a combination of code + instructions. Perhaps `technical_analysis.md` which outlines steps and refers to a script for heavy lifting.

The ClawHub might have ready skills like a `crypto trading` skill or maybe a `stocks` skill; we should check or simply implement our own minimal ones for clarity and control.

1. **Set Heartbeat and Cron:** In `HEARTBEAT.md`, we schedule routine tasks:

   ```
   Every weekday at 9:00 AM:
   - Check pre-market conditions using news_summary and stocks tools for key tickers.
   - Summarize the outlook and decide if any orders should be placed at market open.
   Every hour between 9:30 AM and 4:00 PM:
   - Check portfolio P/L and risk metrics. If any position exceeds threshold (use risk skill), rebalance.
   At 4:00 PM:
   - Summarize daily performance.
   ```

   OpenClaw uses a cron-like syntax in HEARTBEAT (or a simpler interval). For example, one could put a plain instruction "Every 30 minutes: [do X]". The agent will wake up on that schedule and execute the instructions without user prompt [16]. This is extremely useful for continuous trading monitoring.

2. **Multi-Agent Coordination (Optional):** If we want multiple agents as separate entities, we could spin up multiple OpenClaw sessions:

3. Perhaps run one OpenClaw instance with a dedicated config for an Analyst agent, and another for a Trader agent. But having them talk is not trivial unless they communicate via some channel (like one agent could send a message to the other via an API or shared file).
4. A simpler way using one OpenClaw instance: **multi-session** feature can isolate contexts if multiple chat channels are connected [58]. For example, if you connect both a Slack channel and a Telegram channel, each could map to a different persona agent as per config. But then linking them is complex.

5. The approach from the Reddit example was: run one OpenClaw with a Telegram group that includes the same agent multiple times in different roles (clones) or mentions. Actually, the user had one OpenClaw installation that responded to a group chat with three “virtual agents” by distinguishing triggers (like if message contains certain keywords or mentions). They configured one main assistant that delegates tasks to two others by name [60]. This is advanced usage and requires careful prompt engineering so that the single LLM can partition tasks. In our case, we might avoid this and rely on skills instead of separate agents to simplify.

6. **Testing and Iteration:** Start the OpenClaw agent and monitor on the web dashboard. Test that the skills work by manually prompting the agent (via the dashboard or chat) something like “Run technical analysis for AAPL” and see if it uses the `stocks` tool and returns analysis. Adjust the prompts if it doesn’t use tools as expected (OpenClaw’s agent, being an LLM, decides when to use tools based on how you prompt it and how the tools are described). We might need to provide examples in the prompt or as separate **few-shot examples** (OpenClaw allows you to put exemplars

in the agent config). For instance, give an example query and how the agent should respond using a tool. This helps the agent know it can and should use our skills.

7. **Execution and Deployment:** Once satisfied, we let it run. OpenClaw will keep running as a daemon. We ensure our machine is on during trading hours (or run it on a dedicated server or cloud VM). Since OpenClaw is on-prem by design, deploying to a GCP VM (Ubuntu for example) is an option – just install Node and run it there, perhaps exposing the web UI via an SSH tunnel if needed (but not exposing to internet for security). The agent will now handle trading autonomously: e.g., at 9:00 it will post in the chat or log "Good morning, I see XYZ is up, I plan to buy...", then execute by calling `execute_trade.py`. We should log those actions. We can also set up notifications: OpenClaw can message us on Telegram with updates (since it connects to messaging, it can literally send us a message like a status update, which is a great feature to keep a human informed even if they aren't intervening).

8. **Safety Check:** Implement some guardrails. Possibly use OpenClaw's approval feature for high-risk tools. OpenClaw can require certain tool uses to be approved by the user (either automatically approve all in config or manual). For full autonomy we'd auto-approve, but maybe only after testing thoroughly. Since we want no human in loop for operation, we rely on our risk agent logic within to be the guardrail.

End-to-End Example with OpenClaw

Let's illustrate a possible scenario as it would happen with OpenClaw's agent:

- **Morning Briefing (Heartbeat event):** At 8:30 AM, the HEARTBEAT triggers. The agent "TradingAgent" wakes up and sees instructions to gather pre-market info. It composes an internal plan (LLM reasoning) and decides to use its skills:
- It runs the `news_summary` skill (maybe our skills are designed so that simply by mentioning a trigger word in its own "thoughts", the agent can execute it – OpenClaw uses a format like the agent writing `{{CMD: news_summary.py}}` to invoke a command, which the system then runs, returning output).
- The news_summary skill returns: "Major news: ABC CEO resigned, stock down 3% pre-market. XYZ received FDA approval for new drug, stock up 7% pre-market. Overall market futures flat."
- Next, the agent runs `technical_analysis.py` (perhaps feeding it tickers of interest gleaned from news or a list we predefined). That returns: "ABC technicals: downward momentum, XYZ: breakout above resistance, S&P500: RSI 60, mild uptrend."
- The agent's LLM then synthesizes these inputs and forms a message (which it will either log or send to us if configured): e.g., *"Pre-market update: Notable move in XYZ (+7% on FDA news) – looks like a bullish breakout. ABC is facing leadership turmoil (-3%); might be a short opportunity if momentum continues. I will watch these at the open. No action yet, just monitoring. Portfolio is unchanged for now."*

- It might also internally set itself to pay extra attention to XYZ and ABC today (maybe storing in MEMORY.md that these are today's focus).

- **Market Open Reaction:** Suppose at 9:30, the market opens and indeed XYZ starts surging further. The agent could be configured to receive real-time price alerts (via a skill or perhaps by scheduling a continuous loop). Let's say we set a heartbeat every 5 minutes for checking certain prices. At 9:35,

agent checks and sees "XYZ now +10%". The agent's logic (LLM) might recall that it expected bullishness. It might decide: *"This is a significant jump, could be an entry point but it's already up a lot. However momentum is strong. I will buy a small position."*

- It triggers `execute_trade.py` with parameters to buy, say, 50 shares of XYZ.
- The execute_trade skill calls the broker API and returns "Order executed: Bought 50 XYZ @ $Price."
- The agent then outputs (perhaps in the chat to inform): *"Executed buy 50 XYZ to catch momentum on FDA news."*

- **Intra-day Management:** Later, ABC starts dropping significantly (say -5%). The agent's hourly risk check notices ABC (if in portfolio or just as an opportunity). Perhaps we didn't have ABC in portfolio yet, but the agent's strategy might be to short weak stocks. It decides to initiate a short on ABC:

- Before doing so, it runs a risk calculation skill, if available, or just estimates via prompt: "Check if shorting ABC fits risk." The risk logic could be part of the same agent's thought process (the prompt might include: "Always run a risk assessment before executing trade.").
- The agent decides it's safe (maybe because portfolio has longs and this short will hedge). It executes the short via `execute_trade.py` (with a different argument).
- It announces: *"Shorted 100 ABC at $Price as the stock continues to fall post-CEO resignation. This hedges some market exposure."*

- **Agent Coordination:** All this is done by one agent in this scenario. If we had multiple, say an Analyst agent feeding info to a Trader agent within OpenClaw, the flow would be a bit different. But using one agent with tools covers it – because effectively the LLM behind it is handling multi-step reasoning (OpenClaw leverages the LLM to do the planning and decision, similar to how a single human would think: gather info, then decide, then act).

- **Afternoon Adjustment:** Around 3PM, the Fed meeting outcome is announced (surprise rate hike). Our agent could have a skill to check news or perhaps an event triggers it. It sees market turning red. It decides to close that XYZ position to lock profit and cover the ABC short perhaps. It executes those orders accordingly. It might output: *"Fed news came negative. Closing XYZ long for +5% gain, and closing ABC short with +4% gain to secure profits and reduce overnight risk."*

- **End of Day Report:** At 4:00 PM, the heartbeat triggers end-of-day tasks. The agent gathers the day's trade logs (it could store each trade in MEMORY or an external file). It then produces a summary: *"Today's summary: Realized profit on XYZ and ABC trades. Portfolio now 30% cash, 70% equity. Key events: Fed hike – increased volatility, we reduced exposure. Plan for tomorrow: watch bond yields, possibly reduce tech exposure if downturn continues."* This message might be sent to the Telegram channel for the human owner to read.

This end-to-end story shows OpenClaw acting as a **self-directed trader** – it perceives, decides, acts, and informs. We didn't have to script conversation between multiple agents; instead, we leveraged skills to give it multi-faceted abilities. It's less transparent internally (we don't explicitly see a bull agent arguing with a bear agent, but the single agent's LLM might internally simulate those viewpoints if prompted well). The trade-off is simplicity vs. clarity: OpenClaw's single agent approach is simpler to implement, though we

might lose some of the explicit separation of roles. However, the outcome (profitable autonomous trading) can be achieved.

One powerful aspect illustrated is how OpenClaw's **persistent nature** and scheduling suit trading: you essentially get an AI trader that never sleeps and follows a routine you set. And because it can integrate with real tools easily, it truly "does things, not just chat" [21].

Having looked at AutoGen and OpenClaw – two very different paradigms – we will now consider a more developer-centric approach using **LangChain/LangGraph**, which offers fine-grained control for custom multi-agent logic.

Solution 3: Building with LangChain and LangGraph

Implementation Overview

LangChain is a popular framework that provides components to build LLM applications, such as prompt templates, memory, and "agents" that can use tools. While LangChain initially focused on single-agent tool usage (like a QA bot that can use a calculator), it evolved to support multi-agent scenarios. **LangGraph** is an extension in the LangChain ecosystem that specifically targets orchestrating multiple agents in **graph-based workflows** [25]. Essentially, LangGraph allows developers to define a directed graph where nodes are agents or tools and edges define the flow of information or triggers between them. This is ideal for complex workflows where different agents need to run in sequence or in parallel, and maybe converge results.

Using LangChain/LangGraph for our trading system means we will be writing Python code that explicitly coordinates the agents. This gives us a lot of flexibility – we can, for example, create an agent for each specialist and manually call them in whatever order we want, inspect their outputs, and feed those outputs into the next agent. It's not as automated as AutoGen's conversation management, but we get *control* (which in a high-stakes domain like trading can be an advantage: we might want explicit checks between steps).

Key features LangGraph/LangChain bring: - **Stateful Agents with Memory:** Agents can maintain context across calls. We can give each agent a memory buffer to remember what's been said in its interactions [68]. - **Sequential and Parallel flows:** Using LangGraph, we can set up a sequence like: Node1 (AnalystAgent) -> Node2 (StrategyAgent) -> Node3 (RiskAgent). Or even have parallel branches (like two strategy agents in parallel: bull and bear, then join results). - **Integration with Tools:** LangChain has a library of tool integrations (web search, calculators, Python REPL, etc.). We can leverage these for data fetching and execution. Also, it's easy to integrate custom tools by writing a Python function and wrapping it as a LangChain Tool. - **Monitoring and Tracing:** LangChain provides debugging utilities (and LangSmith platform) to trace agent reasoning steps [25] [69]. This can help monitor the multi-agent process and tune prompts.

One example of LangGraph usage is a customer support bot where one agent might classify the query and another generates an answer [27]. In our case, the pattern might be: 1. **Gather Data Node:** (not exactly an agent, possibly just a function call node) – fetch required market data and news. 2. **Analysis Node(s):** One or multiple agents that take the data and produce analysis. For instance, an agent with prompt "You are a market analyst, output key insights" to produce analysis text. 3. **Decision Node:** Another agent that takes analysis as input (plus any other info) and outputs a trade decision. 4. **Risk Node:** Another agent or function

that evaluates the decision (we can implement risk calculation as a Python function rather than an LLM, since risk checks might be formulaic). 5. **Execution Node:** A function to place trades if approved. 6. **Notification/Report Node:** Possibly an agent to summarize the action, or simply an email/push notification sending function.

LangChain allows linking these either programmatically (just calling them in sequence in Python code) or via its chain definitions (like using a `SequentialChain` or `RouterChain`). LangGraph specifically might let us define this flow more declaratively.

Given that LangGraph is new (2024) and has thousands of stars, it likely provides an API to define such flows. For simplicity, one can also manage the sequence manually: - For example, call `analyst_agent.run(input_data)` -> get output text. - Feed that into `strategy_agent.run(analysis_text)` -> get decision. - Pass decision to risk logic (could be `risk_agent.run(decision_text)` or a custom Python if-else). - If ok, call execution function.

This approach is straightforward to understand and debug, albeit imperative.

Technical diagram (LangChain approach): We can conceive of a diagram where each agent is a box and arrows show the data flow:

```
Market Data ---> [Data Fetch Function] ---> Data (prices,news)
                        |
                        v
Fundamentals Agent --> output fundamental insight \
Technical Agent ----> output technical insight    }--> [Strategy Agent] -> trade
plan
News/Sentiment Agent -> output sentiment analysis /
                                        |
                                        v
                                  [Risk Check Function] -> approved trade or
none
                                        |
                                        v
                                  [Execution Tool] (place order)
```

The above shows parallel fetch by different specialized agents and then a strategy agent combining info. This can be implemented by running those agents in parallel threads or sequentially (depending on speed and model usage concerns; parallel is possible if using separate model calls simultaneously).

Step-by-Step Implementation with LangChain/LangGraph

To implement with LangChain:

1. **Setup Environment:** Install `langchain` and any required LLM provider. For Open Source usage, we might use OpenAI's API (so get an OpenAI API key and install `openai` library). Or we could use a

local model via HuggingFace if needed. Also install `langchain-agents` or `langgraph` if it's separate (the Firecrawl article references LangGraph – possibly it's integrated or a separate package). For safety: `pip install langchain langchain-agent` (hypothetical) or as per LangChain docs.

2. **Initialize LLMs and Agents:** Decide on the LLMs for each agent. We might use the same model (GPT-4 or GPT-3.5) for all, just with different prompts. Or possibly specialized models: e.g., use a finance-tuned model for fundamentals (like BloombergGPT if available via HF, or FinGPT), a code-oriented model for technical (if it will write code). For simplicity, assume GPT-4 for main reasoning roles.

3. Create a `PromptTemplate` for each role if needed, or just define system prompts directly in the LangChain agent.
4. Use LangChain's `LLMChain` or `ConversationalAgent` classes to create agents. For example:

```
from langchain.chat_models import ChatOpenAI
llm = ChatOpenAI(model_name="gpt-4", temperature=0.7)
# Agent for fundamental analysis
fundamental_prompt = "You are an expert financial analyst. Given the
following data about a company and market, provide fundamental analysis
insights.\n{input}\nAnalysis:"
fundamental_chain = LLMChain(llm=llm, prompt=PromptTemplate(...,
template=fundamental_prompt))
```

Or use `initialize_agent` from LangChain to create an agent with tools. But since our tasks are mostly pure analysis (no external tool use by the LLM itself, because we handle data separately), simple chains might suffice.

5. If using LangGraph, we might create nodes something like:

```
from langchain.experimental import GraphAgentExecutor
# define nodes and their relationships
```

(Pseudo-code as exact API is hypothetical.)

6. **Data Gathering Tools:** Use Python functions for data fetch. We can integrate them as LangChain Tools (so that an agent could call them if we built a single agent approach). However, a clearer design: perform data gathering outside the LLM calls to have full control:

7. Use yfinance or an API to get stock prices, use a news API (or web scraping using requests/BeautifulSoup or LangChain's SerpAPI tool for news search).
8. We could create a consolidated input for agents from this data, or feed each agent only what it needs (e.g., the Fundamental agent gets earnings reports and ratios, Technical agent gets price and indicator values, etc.).

9. For example:

```
market_data = get_market_data(tickers=["XYZ","ABC"])
news = get_latest_news(keywords=["XYZ","ABC"])
technicals = compute_technicals(market_data)
input_text = f"Stock XYZ: price {market_data['XYZ']}, fundamentals:
{market_data['XYZ_fundamentals']}, News: {news['XYZ']}... \nSimilar for
ABC..."
```

This input_text could then go to one agent who does everything. But we want multi-agent, so we might split:

- fundamental_agent_input = financial data and news summaries,
- technical_agent_input = price data and indicator values,
- sentiment_agent_input = maybe social media sentiment if we had.

10. **Parallel or Sequential Execution:**

11. We could run the analysis agents in parallel threads to save time. LangChain doesn't directly spawn threads for you, but Python's concurrent futures can. Or simply run sequentially if latency isn't critical.
12. Example:

```
fundamental_insights = fundamental_chain.run(fundamental_data_text)
technical_insights = technical_chain.run(technical_data_text)
news_insights = news_chain.run(news_text)
```

Now combine these:

```
combined_analysis = f"Fundamental: {fundamental_insights}\nTechnical:
{technical_insights}\nNews: {news_insights}"
```

13. Alternatively, one can design a single agent that was fed with tools for getting each type of info and it decides what to get (LangChain agents could do that). But dividing by role can yield more targeted outputs.

14. **Strategy/Decision Agent:** Now take the combined analysis and feed into the decision-making agent.

```
strategy_prompt = "You are a portfolio manager. You have the following
inputs from your analysts:\n{analysis}\nBased on this, suggest trading
actions (if any) with rationale."
strategy_chain = LLMChain(llm=llm, prompt=PromptTemplate(...,
template=strategy_prompt))
trade_plan = strategy_chain.run(combined_analysis)
```

The output `trade_plan` might be a text like: "Recommendation: Buy XYZ, Sell ABC... because...".

15. **Risk Assessment:** For risk, we might avoid using LLM (to minimize uncertainty) and instead implement logic: Parse `trade_plan` to extract the proposed trades. If the strategy agent output is in natural language, we might need an LLM to parse it into structured form (alternatively, we could prompt the strategy agent to output in JSON or a list format – which is often done to easily feed to next step). For example, we could instruct the strategy agent prompt to output something like:

    ```
    Actions:
    - action: BUY, ticker: XYZ, amount: 100
    - action: SELL, ticker: ABC, amount: 50
    Rationale: ...
    ```

    If we have that structured, then:

16. Write a Python function `assess_risk(trades)` that checks each trade against predefined limits (position size, etc.) and maybe calculates an estimated risk metric.
17. If risk is too high, either adjust (e.g., cut quantities) or flag it. We can incorporate an LLM risk agent as well, if we want qualitative risk evaluation (like considering correlations, etc.). Let's say we use a simple function for now.

Example:

```
parsed_plan = parse_trade_plan_to_structured(trade_plan)
risk_ok, risk_report = assess_risk(parsed_plan)
if not risk_ok:
    # adjust plan or drop it
    adjusted_plan = adjust_trades(parsed_plan)
    final_plan = adjusted_plan
    rationale += " (Adjusted for risk: ...)"
else:
    final_plan = parsed_plan
```

If we wanted an LLM to double-check the plan, we could have a risk_agent chain that says "Review the proposed trades for any risks or issues" and see if it adds something, but careful it doesn't hallucinate.

1. **Execution:** If `final_plan` has trades, execute them via code. Use broker API or simulated execution:

    ```
    for trade in final_plan:
        execute_trade_api(trade.action, trade.ticker, trade.amount)
    ```

This could be integrated as a LangChain Tool as well. Actually, we could treat `execute_trade` as a Tool and have the strategy agent call it directly if we built a more autonomous agent. But direct coding is fine.

2. **Output & Reporting:** Use LangChain's memory or just store results. Perhaps have a final agent or chain to summarize what was done:

```
summary_prompt = "Summarize today's trading decisions and rationale:\n" +
combined_analysis + "\nPlan: " + trade_plan + "\nRisk assessment: " +
risk_report
summary = summary_chain.run(summary_prompt)
send_email_or_message(summary)
```

   Or simply print it.

3. **Automation Loop:** To run continuously, we'd embed this into a schedule. Could use Python schedule or cron to call this routine at intervals or on events. Because LangChain code can be invoked on demand, deploying on a server with a scheduler (like an Airflow DAG or cron job) could do the trick. GCP wise, one could run this in a Cloud Function triggered by time (though it might exceed typical time limits; maybe Cloud Run or a VM with a cron).

This is more involved to set up than AutoGen or OpenClaw, but it gives full transparency and control. Also, LangChain being widely used means there's community support and examples for each piece (vector store memory, tool integration, etc.).

End-to-End Example with LangChain

Consider how a trading scenario plays out step-by-step with this approach:

- **Data Fetch Stage:** Our script triggers at 9:00 AM. It calls `get_market_data()` which returns that XYZ is up 5% pre-market, ABC is down 2%, plus some fundamental data (like P/E ratios, recent earnings growth, etc.). `get_latest_news()` returns a summary: "XYZ's drug got approval – big positive news; ABC's CEO scandal – negative press." `compute_technicals()` gives indicator values: "XYZ above 50-day MA, momentum rising; ABC broke below support."

- **Analyst Agents Output:**

- FundamentalAgent gets: "XYZ: P/E 15, good earnings growth; ABC: P/E 8 but facing scandal, possible value trap." It outputs an analysis focusing on fundamentals.
- TechnicalAgent gets: "XYZ price trend up, momentum high; ABC in downtrend." Outputs analysis focusing on charts (e.g., "XYZ in strong uptrend, might continue; ABC bearish pattern").

- NewsAgent gets news text, outputs: "XYZ has a positive catalyst (FDA approval) which likely increases its value; ABC has leadership issues making investors cautious."

- **Strategy Agent Output:** It receives all the above. We might combine them or feed them separately; if combined, the prompt might just have a big text. The agent (LLM) produces: *"Given these factors, I suggest going long on XYZ as its news catalyst and technical momentum indicate further upside. Conversely, initiate a short on ABC, since fundamental and news are negative, and technicals confirm a downtrend. Amount: invest 5% of portfolio in XYZ long, 3% in ABC short. This balances risk."*

- **Risk Check:** Our code parses this suggestion. It sees two trades: Buy XYZ (5% portfolio), Short ABC (3% portfolio). It calculates these are within allowed limits (maybe we allow up to 10% position single name). It also might check correlation – perhaps XYZ and ABC are in different sectors, which is fine. So risk is OK. If it were not (say the agent wanted 50% in one stock), we would adjust down.

- **Execution:** The code calls the broker API for these trades. Orders go through (we assume success).

- **Post-Trade Summary:** We generate a summary: "Executed trades: LONG XYZ (5% portfolio) due to FDA approval catalyst and strong momentum. SHORT ABC (3% portfolio) due to management scandal and downward trend. Risk within limits; positions will be monitored." This summary can be printed or sent via email/Slack to stakeholders.

- **Continuous Loop:** Over the day, we could run a smaller loop every hour that uses the same chain of agents on updated data and maybe decides to close or adjust positions. Or one could incorporate logic for if price hits certain thresholds, triggers an exit. These can be coded in directly (like if XYZ up another 5% by midday, realize profit). LangChain doesn't specifically handle event-driven triggers itself; you rely on your own code or external scheduling.

This example shows the LangChain approach yields a similar outcome (buy XYZ, short ABC) but with the developer writing the orchestration. The advantage is you can easily tweak any part: e.g., change the strategy prompt, use a different model for technical analysis, or add a new agent (maybe an OptionsAgent to suggest an options hedge).

One notable aspect: **LangChain/LangGraph is very flexible** – companies have used it to deploy production agents with monitoring [27]. For instance, integrating with LangSmith would let us record each decision step, which could be invaluable for debugging why an agent recommended a trade. This level of observability might surpass what we get from AutoGen or OpenClaw out-of-the-box.

Now that we've covered three major solutions, let's briefly examine the OpenAI Agents SDK approach, which might offer a more straightforward but less feature-rich path.

Solution 4: Building with OpenAI's Agents SDK

Implementation Overview

OpenAI's Agents SDK (released in 2025) provides a simpler, more streamlined interface to create multi-agent systems with robust tracing and guardrails [28]. While conceptually similar to LangChain (it's a Python library and model-agnostic), the OpenAI Agents SDK is designed to lower the barrier for developers to set up multi-agent workflows by offering a lightweight, opinionated framework. It's compatible with many LLMs (100+

models) and emphasizes ease-of-use – meaning it might not have all the bells and whistles of LangChain, but it gets common tasks done with minimal code [29].

For our trading firm scenario, the OpenAI Agents SDK could allow us to define agents (likely as Python classes or config) and then define a workflow where they interact. The documentation suggests it supports: - **Multi-agent conversations** (so we can have agents message each other, similar to AutoGen). - **Tracing** – built-in logging of interactions, which is good for debugging or compliance. - **Guardrails** – possibly content moderation or ensuring agents don't produce disallowed content (not a big issue in trading context, but maybe ensures they don't stray off task or leak API keys). - It might also have some high-level abstractions like a "Coordinator" to manage multiple agents.

The SDK likely provides an interface something like:

```
from openai_agents import Agent, Conversation

class AnalystAgent(Agent):
    # define behavior or prompt
class TraderAgent(Agent):
    # define behavior or prompt

# instantiate
analyst = AnalystAgent(model="gpt-4", prompt="You are an analyst ...")
trader = TraderAgent(model="gpt-4", prompt="You decide trades ...")

# maybe run a conversation
conv = Conversation(agents=[analyst, trader])
result = conv.run(input="start analysis")
```

This pseudo-code is inspired by what an SDK might do. The actual may differ, but conceptually it allows an easy setup.

One can also combine it with tools. The OpenAI Agents SDK likely integrates with OpenAI's function calling or tool usage. We saw an example where it was combined with Firecrawl (for web content) [70] – they extracted content with Firecrawl and then used the SDK to make that content an agent that can converse.

In our use, we might: - Use one agent as the "Analyst" which given a prompt (like "market status") returns analysis. - Another agent as "Trader" that given analyst's info returns a decision. - Possibly a third "Risk" agent or incorporate risk into trader's agent prompt. - Then out-of-band we execute the trade.

If the Agents SDK supports an event loop or function calls, we might incorporate a custom function for execution that the agent can call as a tool. For instance, define a function `execute_order(action, ticker, amount)` and register it so the Trader agent can call it directly in its output (like function calling feature where the model returns a function call JSON that the SDK executes, then returns result).

The strength of OpenAI's SDK is presumably quick prototyping. For example, to replicate the bull vs bear debate, we could spawn two agents and have them talk within a conversation object easily, rather than

writing our own loop. It's similar to AutoGen's goal, but likely more lightweight (AutoGen is quite feature-rich and a bit complex; OpenAI's might be simpler).

Step-by-Step Implementation with OpenAI Agents SDK

1. **Install and Setup:** We'd install the SDK (e.g., `pip install openai-agents-sdk`). Also, ensure we have API access to chosen models or local model integration. The SDK, being provider-agnostic, might let us plug in OpenAI, Anthropic, or local via a plugin. Let's assume using OpenAI for now with an API key set in environment.

2. **Define Agent Behaviors:** The SDK might allow either subclassing an Agent class or simply passing a prompt and tools. Possibly something like:

   ```
   from openai_agents import OpenAIAgent

   analyst = OpenAIAgent(name="Analyst", system_prompt="You analyze
   markets...",
                         model="gpt-4")
   trader = OpenAIAgent(name="Trader", system_prompt="You decide trades...",
   model="gpt-4")
   ```

   If more configuration is needed (like how they handle memory or termination criteria), we would set that. For example, maybe we specify that the conversation ends when Trader agent responds with a trade decision.

3. **Integrate Tools (if needed):** If we want the Analyst agent to have data, either we feed data via prompt or allow it to call functions:

4. The simpler approach: gather data as we did with LangChain and include it in the user prompt to the conversation. For example: `user_input = f"Market Data:\n{market_summary}\nNews:\n{news_summary}\nNow provide analysis."` Then we start the conversation with that.

5. Or, if SDK supports function calling, register functions:

   ```
   def get_stock_price(ticker): ...
   analyst.register_tool("get_stock_price", get_stock_price,
   description="Gets stock price for ticker")
   ```

   Then in its prompt, encourage usage: "To get data, you can use get_stock_price tool." However, for initial prototyping, feeding data directly might be easier.

6. Trade execution tool: define:

```
def execute_trade(action, ticker, amount):
    # call API
    return "Executed"
trader.register_tool("execute_trade", execute_trade, "Execute a trade
order")
```

If the Trader agent's plan comes as a structured function call, the SDK could pick it up and run `execute_trade`.

7. **Conversation Orchestration:** The SDK likely has a conversation or environment where the agents interact. We create a conversation session with our agents:

```
convo = Conversation([analyst, trader])
```

Then start it with an initial message from, say, a user (which in our case is just a dummy to prompt analyst):

```
convo.send("user", "Please analyze the market and suggest trades.")
convo.run()  # or step through
```

Under the hood, the conversation might do:

8. Analyst receives the user prompt, produces an analysis message.
9. Trader sees analyst's message and user prompt, produces a trade decision message.
10. Possibly it alternates or ends based on roles or a condition. We might have to specify that after Trader responds, we stop.

The OpenAI SDK is designed for multi-step workflows, so it might default to letting them continue until a certain stop message or number of turns.

1. **Tracing and Output:** The SDK likely logs the conversation. We capture Trader's final output (trade decision). The SDK's advantage is we didn't have to manually pipe messages; it did it for us. It may also allow injecting a "controller" agent or an observer to ensure the conversation stays on track.

2. **Execute Trades & Loop:** If using function calling, maybe the trader agent already triggered the `execute_trade` function in the middle of conversation, executing it live and possibly returning the result to the conversation (like "Executed trade successfully"). If not using that, we parse the trader's text output and then call our execution function outside the conversation.

After that, we schedule this to run periodically, similar to previous solutions.

1. **Example Agent Interaction:** The conversation might look like:
2. **Analyst (Agent)**: "Market analysis: XYZ looks strong due to [reasons], ABC looks weak due to [reasons]. I think XYZ could be a buy, ABC a sell."

3. **Trader (Agent)**: "Trade plan: Buy 100 XYZ, short 100 ABC. Rationale: ... (If using function, it might actually output a function call like `execute_trade('BUY','XYZ',100)` for each, which the SDK executes.)" If the Trader just gave text, we then execute those orders.

The SDK's guardrails might ensure the agents don't go off-topic or do something weird (like ensure the output is within some format).

1. **Adding Risk Agent (Optional):** We could add a third agent (RiskAgent) into the conversation. Perhaps sequence: Analyst -> Trader -> Risk. But in a conversation, risk would see both previous messages and then respond. That might be slightly unnatural because risk should come after trader. We could do that by adjusting roles or conversation logic: e.g., user triggers, analyst replies, trader replies with plan, then we send that plan as new user prompt to risk agent or have risk agent in the convo to respond next. Possibly conversation with 3 agents could be tricky without custom logic, but maybe doable:
2. It might be easier to incorporate risk into trader's step (just instruct trader to be mindful of risk, or run a separate function after).

Given time, likely skip separate risk agent in SDK example.

End-to-End Example with OpenAI Agents SDK

- We start a scheduled run at some time. The code collects current data and formulates a user message: "Market state: ... What trades do we make?".
- The conversation is initiated. The Analyst agent sees this and outputs analysis.
- The Trader agent sees the analysis and outputs a decision: e.g., *"I recommend buying 50 XYZ and selling 30 ABC."*
- If we set up function calls, maybe Trader's output is in function call form. The SDK would then call `execute_trade` internally for each suggested trade (if we structured it so).
- The result could be that the trades are executed during the conversation, and Trader might then output something like "Executed trade orders for XYZ and ABC."
- The conversation ends.
- The SDK's tracing would have logged each message and any function call for later review.
- We then have the final plan and can, for instance, log it or notify someone.

The difference with LangChain is subtle here – both involve coding, but OpenAI's SDK might reduce boilerplate around message passing. It also likely integrates some safety features out of the box (like checking for model errors, maybe automatically handling partial functions).

Overall, the OpenAI Agents SDK provides a **quick development path**. It might not be as customizable as LangChain but is effective for straightforward agent dialogues.

Having detailed four approaches, we can summarize some key differences:

- **AutoGen:** Great for multi-agent convos with complex patterns, requires coding but has high-level abstractions, proven in production [44] [45].

- **OpenClaw:** Powerful autonomous agent runtime with minimal coding, lots of built-in tools, but one has to fit problem into its paradigm (persistent single agent with skills). Security and correct prompting are considerations [71] [63].
- **LangChain/Graph:** Maximum flexibility for developers, fine control over each step, larger community support, but more manual effort to orchestrate.
- **OpenAI SDK:** Simplicity and official backing, easier multi-agent integration with guardrails, but perhaps less extensibility than others (and reliant on possibly closed-source components or cloud).

It's worth noting that none of these require a human in the loop for normal operation – we design them to be fully autonomous (though with ability for a human to inspect or intervene if needed, which is prudent).

Next, we will discuss future scopes and ideas to enhance such a multi-agent trading system beyond the current state.

Future Scope and Ideas

The concept of an AI agent trading firm is cutting-edge, and there are many avenues to expand and improve it:

- **Reinforcement Learning and Self-Improvement:** Currently, our agents rely on pre-trained LLMs and static prompts. A future enhancement is to incorporate reinforcement learning from real trading outcomes. For example, using **Deep Reinforcement Learning**, the multi-agent system could run simulations (or even live trades on small stakes) and adjust its strategy agents' behavior based on rewards (profit and risk metrics). One could envisage a loop where at end of each week, the AI agents analyze which decisions were profitable and adjust their internal models or prompt biases accordingly. Some research frameworks already aim to incorporate learning in multi-agent LLM systems for trading [72].

- **More Specialized Agents or Departments:** We identified roles like analysts, trader, risk. Future systems might add even more granularity:

- An *Economic Calendar Agent* that watches for scheduled events (Fed meetings, earnings releases) and preps the team with scenarios.
- A *Compliance Agent* to ensure trades follow regulations (e.g., no trading on specific black-out dates or restricted securities).
- A *Client Interaction Agent* (if this were a fund) to draft reports or respond to investor queries about performance, essentially translating the AI's decisions into narratives for humans.

- Agents handling different asset classes: e.g., a *Crypto Specialist Agent* if the portfolio includes cryptocurrencies, or *Forex Agent* for currencies, each using data and models tuned to those markets.

- **Collaboration Between Multiple AI "Firms":** Pushing the envelope, multiple multi-agent systems could interact in a market-like simulation. One could simulate an entire market of AI-driven firms negotiating and trading with each other. This is somewhat happening already in experiments like the **Moltbook phenomenon** where many AI agents interact in a network [73] [74]. For trading, this could mean testing our AI firm against others to see if it finds an edge or if they converge to similar strategies. It could also open the door to *agent-based market modeling*, studying emergent behavior

when many autonomous trading agents compete – possibly yielding insights into market stability or flash crashes caused by AI.

- **Improved Memory and Knowledge Integration:** As time goes on, the agent system will accumulate a lot of data (market history, its own track record). Incorporating a knowledge base or long-term memory is crucial. We might integrate a vector database to let agents recall similar past situations. For instance, if a certain pattern happened before and the outcome was bad, the system should remember not to repeat the mistake. LangChain and others already support vector stores for memory [5]. Future work could combine this with causal reasoning – e.g., the system could store "when interest rates rose last year, our XYZ trade lost money, so now avoid that." This makes the agents truly learn from experience in a non-forgetting way.

- **Model Upgrades and Custom Models:** Currently we lean on general LLMs (GPT-4, Claude). In future, one could train specialized models for each agent (e.g., a transformer fine-tuned on financial texts for the analyst, another on trading strategies for the trader agent). Open-source LLMs like FinGPT or even smaller domain models might be integrated to reduce dependency on proprietary APIs and lower cost. As model context windows grow (Claude already has >100k context versions), feeding realtime data into models will become even easier, enabling agents to see essentially an entire day's market feed in context if needed [66].

- **Enhanced Toolsets and Real-world Integration:** We can integrate more real-time tools: for example, connect an agent to a **streaming price feed** or a **websocket** from exchanges for instantaneous reaction (right now we rely on periodic polling). Agents could also interface with more complex trading instruments – e.g., automatically constructing and trading options strategies, futures, or using portfolio margin calculations. Incorporating a *Optimization Tool* (like calling a quadratic optimizer for portfolio allocation) as part of the agent's arsenal could improve decisions (the agent could ask the optimizer tool for the best allocation given its forecasts).

- **User Interface & Visualization:** While we focused on back-end functionality (no UI as requested), a future addition is a dashboard for humans to monitor the AI agents. This could show what each agent is "thinking" (their last analysis message), current portfolio positions, and upcoming scheduled actions. Visualization of agent decisions (like a timeline of trades with their reasons) would build trust with users. This doesn't change the agent logic but is important for practical adoption.

- **Robustness and Safety:** Future development should address robustness: ensuring the system handles unexpected inputs or model errors gracefully. For instance, if an LLM agent outputs something nonsensical or the external data API fails, the system should catch that and perhaps retry or ask for human assistance. One might incorporate anomaly detection – a separate meta-agent that observes the others and flags if an agent's output seems out of character or potentially harmful (like suddenly suggesting an all-in bet on a penny stock). This is akin to having safety supervisors for the AI. OpenAI's guardrails and content filters are a start [75] [7], but domain-specific guardrails (like "don't violate risk limits no matter what the LLM says") can be codified.

- **Scale and Distributed Agents:** As we increase the number of agents or complexity, scaling becomes a consideration. Running many agents concurrently could tax compute (especially if each uses a large model). Future systems might use a hierarchy: some agents run with smaller, faster models for continuous monitoring, and only escalate to a big model when needed (e.g., a complex decision).

This hierarchal agent approach can optimize resource use. Also, distributing agents across multiple machines or cloud functions could be explored (with a message bus connecting them). CrewAI or LangGraph concepts might evolve to orchestrate agents distributed in the cloud.

- **Inspiration from Real Firms:** We can continue to refine agents by studying how top human traders operate. For example, incorporate a *Scalping Agent* if one wants high-frequency trading elements, or a *Machine Learning Quant Agent* that perhaps runs a traditional quant model (like a random forest on price data) and feeds its output into the discussion. The future AI trading firm might blend symbolic AI (rules, math) and LLM-based reasoning for best results – leveraging the precision of algorithms and the adaptability of LLMs.

In conclusion, the future is bright (and perhaps a bit daunting) for autonomous trading agent systems. With frameworks like AutoGen and OpenClaw maturing and new ones emerging rapidly, we can expect more **"AI employees"** handling sophisticated tasks [76]. As one article noted, these agent platforms are becoming part of how businesses operate, quietly running in the background of daily operations [77]. In finance, where information and reaction speed are paramount, a well-designed farm of AI agents could become a powerful edge. The ideas above will help ensure such a system stays ahead of the curve, learning, adapting, and ultimately performing in an ever-changing market landscape.

---

[1] [2] [3] [9] [10] [11] [12] [18] [19] Your Guide to the TradingAgents Multi-Agent LLM Framework | DigitalOcean
https://www.digitalocean.com/resources/articles/tradingagents-llm-framework

[4] [15] [16] [21] [22] [53] [54] [55] [56] [57] [58] [59] [63] [64] [65] [66] [67] [71] [73] [74] What Is OpenClaw? The Open-Source AI Agent That Actually Does Things | MindStudio
https://www.mindstudio.ai/blog/what-is-openclaw-ai-agent/

[5] [20] [25] [26] [27] [28] [29] [30] [31] [32] [33] [34] [68] [69] The Best Open Source Frameworks For Building AI Agents in 2026
https://www.firecrawl.dev/blog/best-open-source-agent-frameworks

[6] [23] [24] [35] [60] [61] [62] I built 4 OpenClaws in 4 hours - here's the architecture and results : r/SideProject
https://www.reddit.com/r/SideProject/comments/1r2mbai/i_built_4_openclaws_in_4_hours_heres_the/

[7] [8] [75] [76] [77] OpenClaw and the Observability Gap in Autonomous AI Agents | Fiddler AI Blog
https://www.fiddler.ai/blog/openclaw-ai-observability-gap

[13] [14] [39] [51] Customize Speaker Selection | AutoGen 0.2
https://microsoft.github.io/autogen/0.2/docs/topics/groupchat/customized_speaker_selection/

[17] [37] [46] [47] [48] [49] [50] [52] Getting Started | AutoGen 0.2
https://microsoft.github.io/autogen/0.2/docs/Getting-Started/

[36] [38] [40] [41] [42] [43] [44] [45] AI Agents with AutoGen — OpenClaw | Mark Cijo — AI Agents Expert
https://markcijo.ai/built-with/autogen

[70] Converting Entire Websites into Agents with Firecrawl's LLMs.txt Endpoint and OpenAI Agents SDK
https://www.firecrawl.dev/blog/website-to-agent-with-firecrawl-openai

[72] [PDF] TradingAgents: Multi-Agents LLM Financial Trading Framework
https://arxiv.org/pdf/2412.20138